

光线追踪大作业实验报告

基础光线生成与 BVH 加速渲染

姓名：_____ 学号：_____

2026 年 6 月 22 日

摘要

本次大作业基于课程提供的 C++ 光线追踪框架, 完成了一个 Whitted 风格的光线追踪渲染器。具体实现了四个核心函数: `Renderer::Render` (主光线生成)、`Triangle::getIntersection` (光线-三角形求交, Möller-Trumbore 算法)、`Bounds3::IntersectP` (光线-包围盒求交, slab 测试)与 `BVHAccel::getIntersection` (BVH 层次遍历加速求交); 并额外完成了 `Scene::castRay_noBVH` 中的完整着色模型 (加分项)。在 1280×960 分辨率、含 **4968** 个三角形的 Stanford Bunny 场景下, BVH 模式约 **0.45 s** 即可渲染出正确的着色图像。在此基础上, 本文进一步新增逐三角形暴力求交路径, 并通过自动化实验对比三个不同面数的 Stanford 模型 (Dragon、Armadillo) 在 BVH 与无 BVH 下的渲染耗时, BVH 加速比最高达 **8364** 倍。本文逐一说明各函数的算法原理、关键代码与实验结果。

1 概述

光线追踪通过**逆向**追踪从相机出发、穿过每个像素射入场景的光线, 求出光线与场景的最近交点并在该点着色, 从而生成图像。当场景中三角形数量较大时, ”对每条光线遍历所有三角形”的暴力做法效率极低; 为此需要使用 **层次包围盒 (Bounding Volume Hierarchy, BVH)** 对求交过程进行加速: 先用廉价的光线-包围盒测试快速排除大片不可能相交的空间, 再递归深入到真正可能相交的少数三角形。整体渲染流程如图 1 所示。

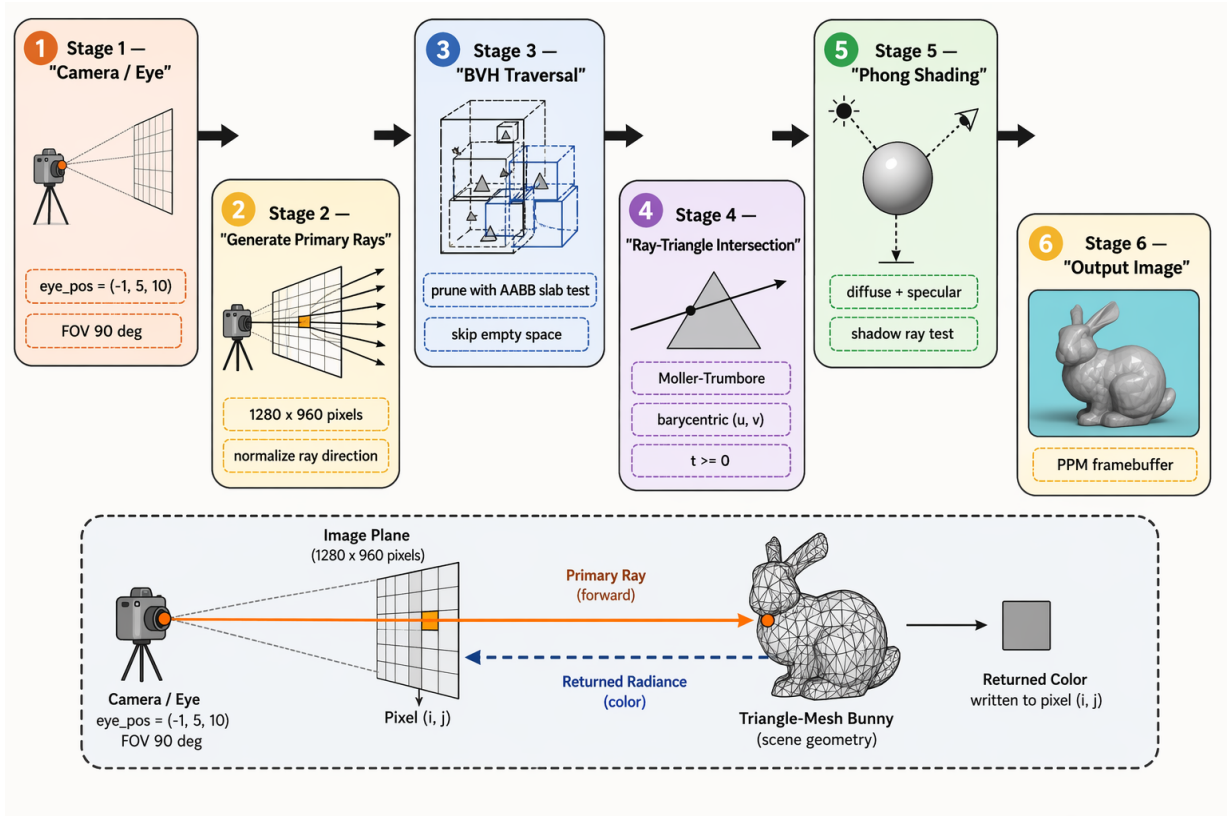


图 1: 光线追踪渲染流程: 相机生成主光线 → BVH 遍历剪枝 → Möller-Trumbore 光线-三角形求交 → Phong 着色 (含阴影测试) → 写入帧缓冲并输出 PPM 图像。

本框架需要在以下文件中补全代码: `Renderer.cpp`、`Triangle.hpp`、`Bounds3.hpp`、`BVH.cpp`, 并可选地在 `Scene.cpp` 中补全着色模型。下面分别说明。

2 各函数的实现

2.1 主光线生成: `Renderer::Render`

该函数遍历图像的每个像素 (i, j) , 构造一条从相机出发、穿过该像素中心的光线。设视场角为 fov , 则 $\text{scale} = \tan(\text{fov}/2)$, 图像宽高比 $\text{aspect} = \text{width}/\text{height}$ 。将像素坐标映射到位于 $z = -1$ 平面上的相机空间坐标:

$$x = (2 \cdot \frac{i+0.5}{\text{width}} - 1) \cdot \text{aspect} \cdot \text{scale}, \quad (1)$$

$$y = (1 - 2 \cdot \frac{j+0.5}{\text{height}}) \cdot \text{scale}. \quad (2)$$

其中 $+0.5$ 使光线穿过像素中心, y 方向取负是因为图像行号自上而下增大而相机 y 轴向上。随后归一化方向并投射光线:

```
1 float x = (2.0f * (i + 0.5f) / scene.width - 1.0f) * imageAspectRatio * scale;
2 float y = (1.0f - 2.0f * (j + 0.5f) / scene.height) * scale;
3 Vector3f dir = normalize(Vector3f(x, y, -1));
4 Ray ray(eye_pos, dir);
5 framebuffer[m++] = check_mode ? scene.castRay_noBVH(ray, 0) : scene.castRay(ray, 0);
```

最终把浮点颜色 clamp 到 $[0, 1]$ 并按 P6 格式写出 `binary.ppm`。

2.2 光线-三角形求交: `Triangle::getIntersection` (Möller-Trumbore)

光线 $\mathbf{O} + t\mathbf{D}$ 与三角形 $(\mathbf{P}_0, \mathbf{P}_1, \mathbf{P}_2)$ 的交点可用重心坐标 (u, v) 表示:

$$\mathbf{O} + t\mathbf{D} = (1 - u - v)\mathbf{P}_0 + u\mathbf{P}_1 + v\mathbf{P}_2. \quad (3)$$

令 $\mathbf{E}_1 = \mathbf{P}_1 - \mathbf{P}_0$, $\mathbf{E}_2 = \mathbf{P}_2 - \mathbf{P}_0$, $\mathbf{T} = \mathbf{O} - \mathbf{P}_0$, 由 Cramer 法则 (Möller-Trumbore) 可得

$$\begin{bmatrix} t \\ u \\ v \end{bmatrix} = \frac{1}{\mathbf{E}_1 \cdot (\mathbf{D} \times \mathbf{E}_2)} \begin{bmatrix} \mathbf{E}_2 \cdot (\mathbf{T} \times \mathbf{E}_1) \\ \mathbf{T} \cdot (\mathbf{D} \times \mathbf{E}_2) \\ \mathbf{D} \cdot (\mathbf{T} \times \mathbf{E}_1) \end{bmatrix}. \quad (4)$$

有效交点必须满足 $u \geq 0$, $v \geq 0$, $u + v \leq 1$ (落在三角形内) 且 $t \geq 0$ (位于光线前方)。实现上:

```
1 // 背面剔除: 只对正对相机的面求交, 避免封闭网格内壁干扰
2 if (dotProduct(ray.direction, normal) > 0) return inter;
3
4 Vector3f pvec = crossProduct(ray.direction, e2);
5 float det = dotProduct(e1, pvec);
6 if (fabs(det) < EPSILON) return inter; // 光线与三角形平行
7
8 float invDet = 1.0f / det;
9 Vector3f tvec = ray.origin - v0;
10 float u = dotProduct(tvec, pvec) * invDet;
11 if (u < 0 || u > 1) return inter;
12 Vector3f qvec = crossProduct(tvec, e1);
13 float v = dotProduct(ray.direction, qvec) * invDet;
14 if (v < 0 || u + v > 1) return inter;
15 float t = dotProduct(e2, qvec) * invDet;
16 if (t < 0) return inter; // 交点在光线起点之后
17
18 inter.happened = true; inter.coords = ray(t);
19 inter.normal = normal; inter.distance = t;
20 inter.obj = this; inter.m = m;
```

实现要点: 相比框架自带的辅助函数, 这里在函数内部直接实现了 Möller-Trumbore, 并补充了两处健壮性判断——平行判定使用 $|det| < \varepsilon$ (而非严格等于 0), 以及 $t \geq 0$ 剔除位于相机背后的交点, 避免错误交点污染 BVH 的最近距离比较。

2.3 光线-包围盒求交: `Bounds3::IntersectP` (slab 测试)

轴对齐包围盒 (AABB) 可视为三对相互垂直的平行平面 (slab)。光线进入第 k 个 slab 的参数为 $t_k^{\min} = (p_k^{\min} - O_k)/D_k$, 离开为 $t_k^{\max} = (p_k^{\max} - O_k)/D_k$ 。光线进入整个盒子的时刻为各轴进入时刻的最大值, 离开盒子的时刻为各轴离开时刻的最小值:

$$t_{\text{enter}} = \max_k t_k^{\min}, \quad t_{\text{exit}} = \min_k t_k^{\max}. \quad (5)$$


```

4         int(ray.direction.z > 0) };
5 if (!node->bounds.IntersectP(ray, invDir, dirIsNeg)) return Intersection(); // 剪枝
6 if (node->left == nullptr && node->right == nullptr)
7     return node->object->getIntersection(ray); // 叶节点
8 Intersection hit1 = getIntersection(node->left, ray);
9 Intersection hit2 = getIntersection(node->right, ray);
10 return hit1.distance < hit2.distance ? hit1 : hit2; // 取更近者

```

由于 Intersection 的默认 distance 为 double 最大值，未命中的子树自然不会赢得最近距离比较，因此无需额外的命中判断即可正确合并左右结果。

2.5 额外内容：castRay_noBVH 的完整着色

框架在检测模式 (./RayTracing check) 下不会调用 buildBVH(), 故 scene.bvh 为空，原 castRay_noBVH 仅返回纯白以验证求交是否正确。本部分按文档“额外内容”要求，参照上方的 castRay 补全了完整的 Whitted 着色模型。为不依赖场景级 BVH，新增了暴力最近交点函数：

```

1 Intersection Scene::intersect_noBVH(const Ray &ray) const {
2     Intersection nearest; // 默认 distance = DBL_MAX
3     for (uint32_t k = 0; k < objects.size(); ++k) {
4         Intersection it = objects[k]->getIntersectionNoBVH(ray); // 真.逐三角形暴力
5         if (it.happened && it.distance < nearest.distance) nearest = it;
6     }
7     return nearest;
8 }

```

这里的 getIntersectionNoBVH 对网格物体直接遍历其全部三角形、绕过 MeshTriangle 内部 BVH，从而提供真正不含任何层次结构的暴力求交基线（用于后文的加速实验）。castRay_noBVH 在此基础上完成与 castRay 一致的三支着色：（1）反射 + 折射材质用 Fresnel 方程混合反/折射颜色；（2）镜面反射材质追投反射光线；（3）漫反射/光泽材质使用 Phong 模型（漫反射 + 高光），并对每个光源投射阴影光线判断遮挡：

$$L = \sum_{\text{lights}} (1 - \text{inShadow}) I (\mathbf{N} \cdot \mathbf{l}) k_d c_d + I \max(0, -\mathbf{r} \cdot \mathbf{d})^p k_s, \quad (6)$$

其中阴影测试与递归光线均改用 intersect_noBVH。Bunny 材质为漫反射/光泽，故走 Phong 分支，最终检测模式得到与 BVH 模式逐像素完全一致的着色图像。

3 实验结果

3.1 渲染正确性验证 (Stanford Bunny)

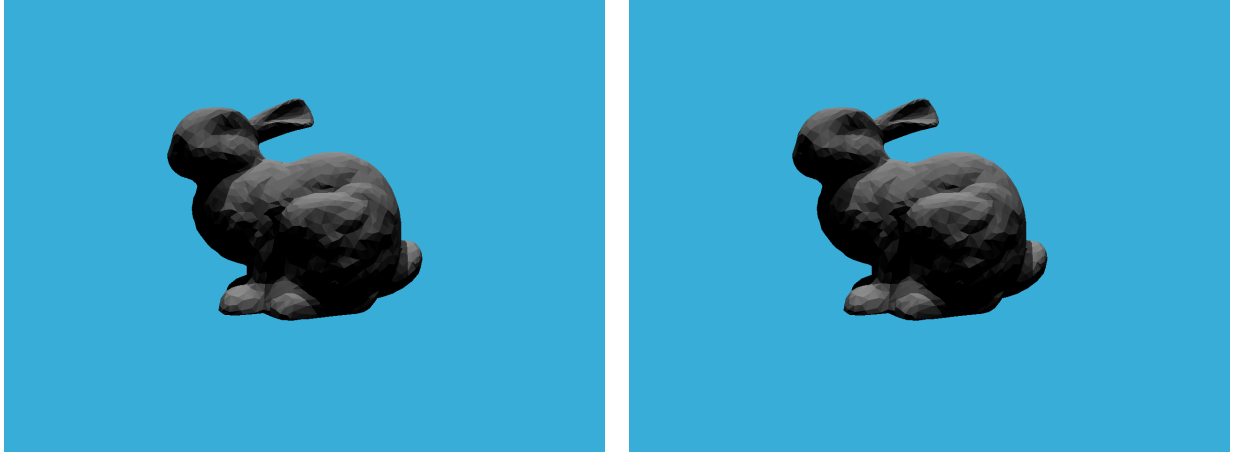
编译与运行：

```

mkdir build && cd build && cmake .. -DCMAKE_BUILD_TYPE=Release && make
./RayTracing # BVH 加速渲染 -> binary.ppm
./RayTracing check # 检测/着色模式 -> binary.ppm

```

两种模式的渲染结果如图 3。BVH 模式（左）正确渲染出带漫反射与高光、含自阴影的灰色兔子；检测模式（右）在补全着色模型后得到与之一致的图像，验证了求交与着色逻辑的正确性。



(a) BVH 加速渲染 (./RayTracing)

(b) 检测/着色模式 (./RayTracing check)

图 3: 1280×960 、4968 三角形的 Stanford Bunny 渲染结果。两图逐像素一致。

在 1280×960 分辨率下，Bunny 场景的 BVH 模式约 0.45s 即可完成渲染，且两种模式输出**逐像素一致**，验证了求交与着色逻辑的正确性。

需要说明的是，MeshTriangle 在构造时已对其内部 4968 个三角形建立了一层 BVH；若检测模式仍调用常规 `getIntersection`，则即便关闭**场景级** BVH，**网格内部**的 BVH 仍在发挥加速作用，无法得到真正的“无 BVH”基线。为获得公平对照，本文新增逐三角形暴力求交路径 `getIntersectionNoBVH`（绕过网格内部 BVH、直接遍历全部三角形），并在 `main.cpp` 中输出高精度的 `Render seconds` 计时，配合自动化实验脚本 `tools/run_experiments_ubuntu.sh`，在不同面数模型上系统评估 BVH 的加速收益，如下节所示。

3.2 不同模型面数下的 BVH 加速效果分析

为进一步验证 BVH 对光线-三角形求交过程的加速效果，本文选取三个不同面数的 Stanford 模型进行对比实验。实验分辨率均为 1280×960 ，渲染内容和光照设置保持一致，仅替换输入网格模型。对每个模型分别测试两种模式：一种为启用 BVH 的正常渲染模式，另一种为关闭 BVH 后逐三角形暴力遍历求交的 `check` 模式（经 `getIntersectionNoBVH` 直接遍历网格全部三角形、不依赖任何层次结构）。整个流程由自动化脚本 `run_experiments_ubuntu.sh` 在 Release 构建下完成：对每个模型依次运行两种模式、解析程序输出的高精度 `Render seconds` 计时并汇总为 `summary.csv`，保证测量一致、可复现。实验结果如表 1 与图 4 所示。

表 1: 不同模型面数下 BVH 与暴力求交渲染时间对比

模型	顶点数	面数	BVH 渲染时间/s	无 BVH 渲染时间/s	加速比
Dragon res3	22998	47794	0.394	538.566	1368.37
Dragon res2	100250	202520	0.549	2535.280	4616.81
Armadillo	172974	345944	0.601	5029.130	8364.03



图 4: 三个不同面数 Stanford 模型在 BVH 模式下的渲染结果 (1280×960), 对应表 1 中 BVH 列的输出。三幅图像均带有正确的漫反射、高光与自阴影; 随面数增加, 模型表面细节愈发丰富 (对比 Dragon res3 与 res2 的鳞片与轮廓), 而 BVH 渲染耗时仍稳定在亚秒级。

从实验结果可以看出, BVH 对三角网格渲染具有非常显著的加速效果。对于面数最少的 Dragon res3 模型, 暴力求交需要 538.566 s, 约为 8.98 分钟; 启用 BVH 后仅需 0.394 s, 加速比达到 1368.37 倍。随着模型面数增加, 加速效果进一步增强: Dragon res2 的面数约为 Dragon res3 的 4.24 倍, 暴力求交时间增加到 2535.280 s, 而 BVH 渲染时间仅增加到 0.549 s; Armadillo 模型包含 345944 个三角形, 暴力求交时间达到 5029.130 s, 约为 83.82 分钟, 而 BVH 模式仍能在 0.601 s 内完成渲染。

造成这一现象的主要原因是, 暴力求交在每条光线与场景相交时都需要遍历模型中的全部三角形, 其计算量随三角形数量近似线性增长。当图像分辨率固定为 1280×960 时, 主光线数量已经超过一百万条, 阴影光线还会进一步增加求交次数, 因此高面数模型下暴力求交的时间会迅速上升。相比之下, BVH 将三角形组织成层次包围盒结构, 光线首先与包围盒进行相交测试, 只有通过包围盒测试的少量节点才会继续向下遍历。大量不可能相交的三角形会被提前剪枝, 因此单条光线实际访问的三角形数量大幅减少。

从增长趋势上看, 模型面数从 47794 增加到 345944, 约增加 7.24 倍; 无 BVH 模式的渲染时间从 538.566 s 增加到 5029.130 s, 约增加 9.34 倍, 表现出对面数非常敏感的特征。而 BVH 模式下渲染时间仅从 0.394 s 增加到 0.601 s, 增长约 1.53 倍, 说明 BVH 能够有效削弱模型复杂度对渲染时间的影响。加速比也随着面数增加而明显提升, 从 1368.37 倍提高到 8364.03 倍, 说明模型越复杂, BVH 的剪枝收益越明显。

综上, 本实验表明: 在三角网格数量较大时, 直接逐三角形求交会使得渲染时间达到分钟甚至小时级别, 难以满足实际使用需求; 而 BVH 能够将同样场景的渲染时间稳定控制在秒级以内。对于复杂模型, BVH 不仅显著降低了单次渲染耗时, 也使高面数模型的光线追踪渲染具有可行性。

4 总结

本次作业完整实现了基于 BVH 加速的 Whitted 风格光线追踪器: 从主光线生成、Möller-Trumbore 光线-三角形求交、AABB slab 包围盒测试, 到 BVH 递归遍历, 并额外补全了非 BVH 路径下的完整着色模型。在四千余三角形的 Bunny 场景上以约 0.45 s 渲染出正确结果。进一步的跨模型对照实验表明, 相比逐三角形暴力求交, BVH 在 4.8 万至 34.6 万面的 Stanford 模型上取得 1368 至 8364 倍加速, 且渲染耗时几乎不随面数增长 ($0.39 \rightarrow 0.60$ s), 凸显了层次包围盒在复

杂场景下的必要性。通过本次实践，加深了对光线生成、解析法求交、空间数据结构加速以及局部光照（Phong + 阴影）等图形学核心概念的理解。